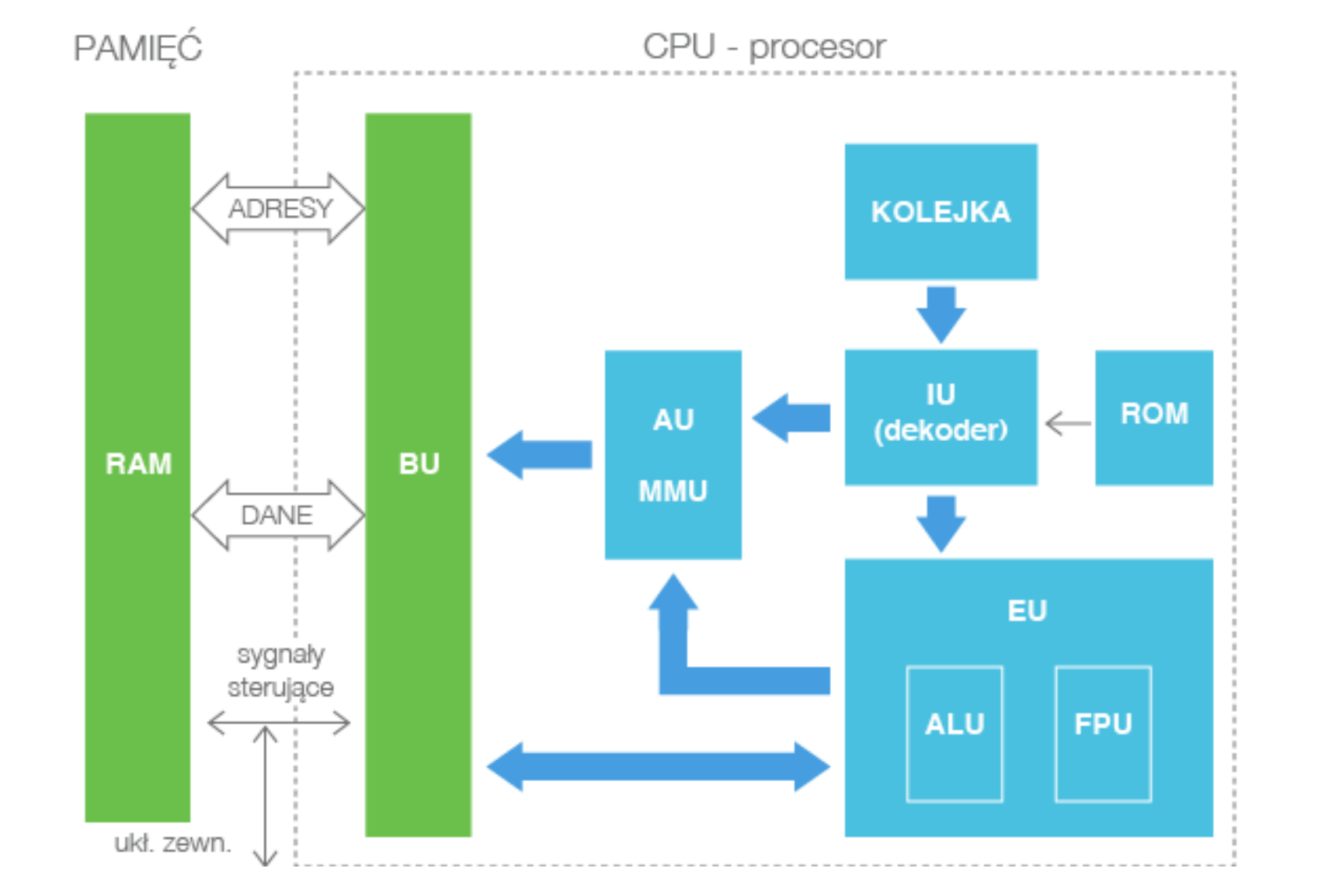
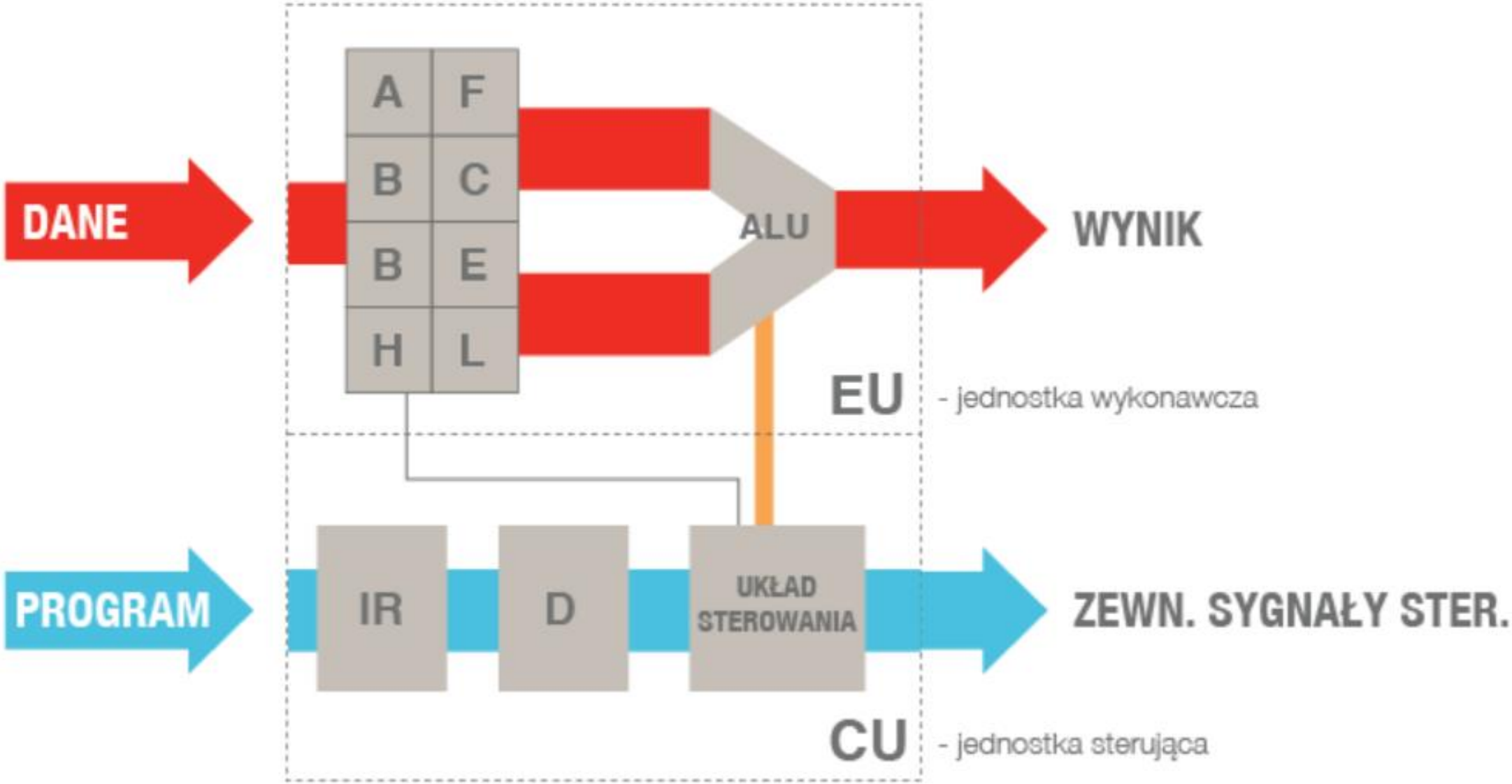
Część 1

Asembler

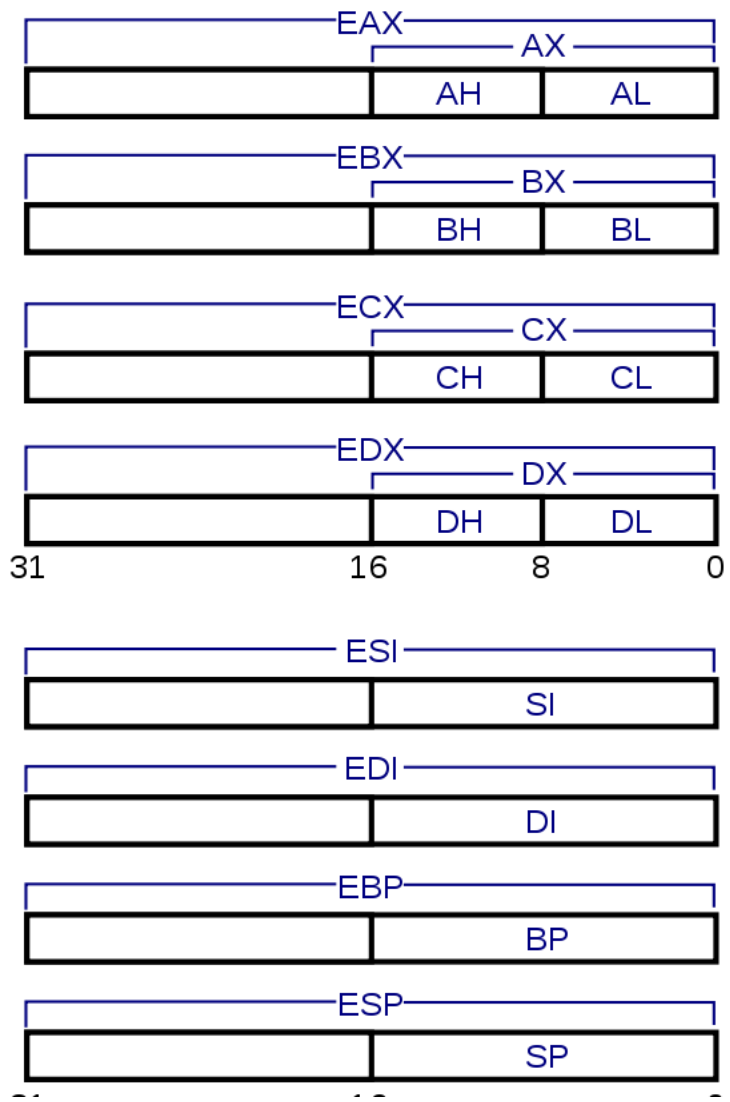


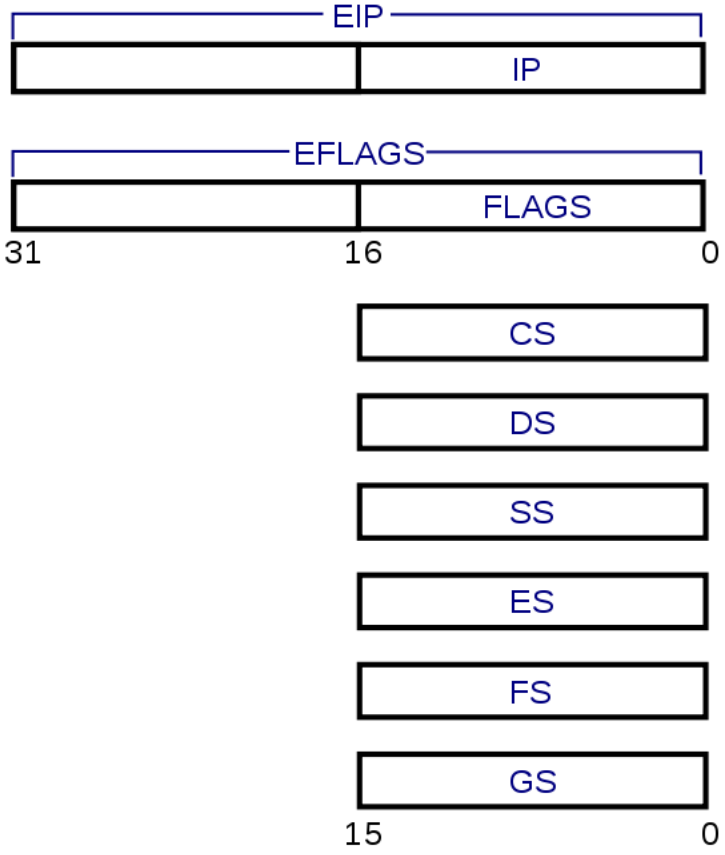
Procesor

Źródło: https://lenovozone.pl/blog/poznajmy-technologie-jak-dziala-procesor/



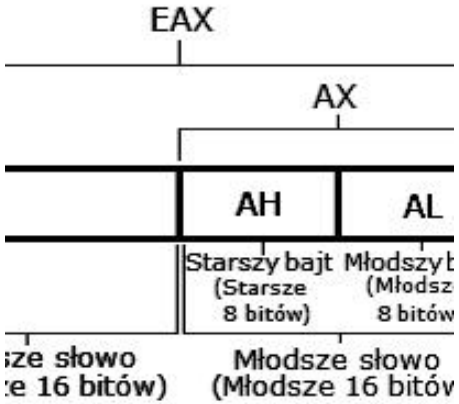
Źródło: https://lenovozone.pl/blog/poznajmy-technologie-jak-dziala-procesor/





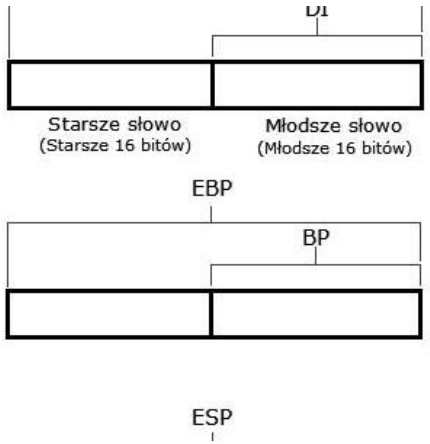
Źródło: wikipedia

Rejestry procesora



Rejestry procesora

- EAX (Akumulator) – do obliczeń arytmetycznych/logicznych
- EBX (Rejestr bazowy) – do adresowania
- ECX (Rejestr licznika) – np. jako licznik w pętlach
- EDX (Rejestr danych) – np. do przechowywania adresów zmiennych



Rejestry indeksowe

- EDI rejestr przeznaczenia
- ESI rejestr źródła
- dzielą się na DI i SI
- **Ważne:** Jeżeli modyfikujemy zawartość rejestrów ESI, EDI, EBP lub ESP musimy je przywrócić zanim wrócimy do Windows.
- EBP wskaźnik bazowy
- ESP wskaźnik stosu

Rejestry flagi

- **CF (Carry Flag) – znacznik przeniesienia.** Flaga zostaje ustawiona, gdy podczas działania nastąpiło przeniesienie z bitu najbardziej znaczącego poza dostępny zakres zapisu. W przeciwnym wypadku znacznik ma wartość zero.
- **PF** (Parity Flag) – znacznik parzystości. Ustawiany jest, gdy w wyniku wykonania działania liczba bitów o wartości 1 w mniej znaczącym bajcie jest parzysta.
- **AF** (Auxillary Flag) – znacznik przeniesienia pomocniczego. Ustawiany, gdy nastąpiło przeniesienie z bitu numer 3 na bit numer 4 lub pożyczenie z bitu numer 4.
- **ZF (Zero Flag) – znacznik zerowy**. Przyjmuje wartość 1, gdy wynik działania ostatnio wykonanego dał w wyniku zero.
- **SF (Sign Flag) – znacznik zmiany znaku.** Ustawiany jest, gdy najbardziej znaczący bit (inaczej: najstarszy bit, nazywany bitem znaku) w otrzymanym wyniku równa się jeden.
- **OF (Overflow Flag) – znacznik nadmiaru/przepełnienia**. Ustawiany jest, gdy w danym działaniu nastąpiło przeniesienie na najstarszy bit, lub nastąpiło pożyczenie z tego bitu.
- **TF** (Trap Flag) – ustawia się przy debugowaniu. Procesor wywołuje wtedy tylko jedną instrukcję, a następnie wywołuje przerwanie, co pozwala dołączonego debuggerowi zbadać program.
- **IF** (Interrupt Flag) – zezwolenie na przerwanie.
- **DF** (Direction Flag) – znacznik kierunku.

Rejestry flagi

- **IOPL** – I/O privilege level (286+ only). Informuje o priorytecie dostępu do portów Wejścia-Wyjścia. CPL (Current Priority Level) musi być większe lub równe od tego znacznika, aby program miał dostęp do portów Wejścia-Wyjścia.
- **NT** – Nested task flag (286+ only).
- **RF** – Resume flag (386+ only). Znacznik wznowienia.
- **VM** – Virtual 8086 mode flag (386+ only). Włączenie trybu wirtualnego.
- **AC** – Alignment check (486SX+ only).
- **VIF** – Virtual interrupt flag (Pentium+).
- **VIP** – Virtual interrupt pending (Pentium+).
- **ID** – Identification (Pentium+). Odczyt rodzaju procesora.

```
_asm mov eax, 25; //eax przyjmuje wartość 25
_asm mov ecx, eax; //skopiuj do ecx wartość z eax
_asm mov zmienna, eax;// skopiuj do zmiennej wartość z eax
```

Podstawowe instrukcje

- **MOV** (kopiuj)
- **Składnia**: MOV destination, source

Instrukcje arytmetyczne

```
_asm mov eax, i;
_asm mov ebx, j;
_asm add eax, ebx;
```

- **ADD** - dodawanie

Składnia: **ADD** operand1, operand2

Instrukcja **ADD** dodaje wartość do rejestru lub komórki pamięci.

- **SUB** - odejmowanie

Składnia: **SUB** operand1, operand2

Odejmuje wartość drugiego operandu od pierwszego i wynik zapisuje w pierwszym.

Instrukcje arytmetyczne

- **MUL** mnożenie

Składnia: **MUL** operand1

Mnoży rejestr eax (lub ax, lub al) przez podaną wartość. Mnoży liczby bez znaku.

- **DIV** dzielenie

Składnia: **DIV** dzielnik

Instrukcja ta dzieli wartość w rejestrze eax przez wartość podaną jako dzielnik. Dzielona jest zawsze wartość w eax, wynik też będzie w eax (al). Reszta z dzielenia natomiast w edx (ah).

Mnożenie ze znakiem

- **IMUL**

Składnia: IMUL wartość
IMUL wynik, wartość, wartość
IMUL wynik, wartość

Mnoży wartość z eax z podaną wartością (IMUL wartość), lub mnoży dwie wartości i wynik zachowuje w pierwszym operandzie (IMUL wynik, wartość, wartość), lub mnoży rejestr z wartością (IMUL wynik, wartość).

Dzielenie ze znakiem

- **IDIV**

Składnia: **IDIV** dzielnik

Działa podobnie jak instrukcja div, tylko, że dzieli ze znakiem.

Inkrementacja i dekrementacja

- **INC** (inkrementacja)

Składnia: **INC** rejestr lub zmienna

Działa odwrotnie do instrukcji DEC, zwiększa wartość rejestru lub zmiennej o 1.

- **DEC** (dekrementacja)

Składnia: **DEC** rejestr lub zmienna

Porównanie wartości

- cmp

Składnia: **cmp** operand1, operand2;

Dodatkowe informacje:

https://pl.wikibooks.org/wiki/Asembler_x86/Instrukcje/Skokowe

Instrukcje logiczne:

- AND

Składnia: **and** operand1, operand2

- OR

Składnia: **or** operand1, operand2

- XOR

Składnia: **xor** operand1, operand2

- NOT

Składnia: **not** operand1

Instrukcja skoku

- **Skok bezwarunkowy**

Etykieta:

jmp Etykieta;

- **Skok warunkowy**

jg – skacz gdy wynik operacji jest dodatni (SF wyzerowana)

je – skacz gdy wynikiem operacji jest zero (ZF ustawiony)

jge – skacz gdy wynik operacji jest więky lub równy zero (SF wyzerowany)

jl – skacz gdy wynik był mniejszy od zera (SF ustawiony)

jle - skacz gdy wynik był mniejszy lub równy od zero (ZF lub SF ustawiony)

jne – skacz gdy wynik był różny od zera (ZF wyzerowany)

…

Zadania - instrukcje

- Niech będzie dana zmienna i typu int. Napisz wstawkę asemblerową przypisującą zmiennej i wartość 10. Użyj rejestru eax.
- Niech i, j, k będą zmiennymi typu int. Napisz wstawkę asemblerową wykonującą operację k=i+j. Użyj rejestrów eax i ebx.
- Niech i, j, k będą zmiennymi typu short int. Napisz wstawkę asemblerową wykonującą operację k= (i+j) % 256.
- Niech i, k będą zmiennymi typu short int. Napisz wstawkę asemblerową przypisującą do zmiennej k sumę starszego i młodszego bajtu zmiennej i.
- Niech i będzie zmienną typu int. Napisz wstawkę asemblerową sprawdzającą czy liczba jest równa zero.
- Niech i, j będą zmiennymi typu int. Napisz wstawkę asemblerową sprawdzającą która liczba jest większa.
- Niech i, j, k będą zmiennymi typu short int. Napisz wstawkę asemblerową podstawiającą do zmiennej k wartość 0 jeżeli i<j, wartość 1 jeżeli i==j, wartość 2 jeżeli i>j.